

Projeto 3: Otimizando Estratégias de E-Commerce com Análise de Dados

Contextualização

O setor de **e-commerce** cresce rapidamente, mas enfrenta desafios na retenção de clientes, aumento do ticket médio e redução de cancelamentos. Empresas que vendem online precisam entender melhor o comportamento dos consumidores para criar estratégias de marketing mais eficientes e personalizadas.

Neste projeto, você atuará como um analista de dados contratado por uma loja de e-commerce para analisar transações passadas e identificar padrões que possam ajudar a empresa a **melhorar as vendas, segmentar clientes e reduzir cancelamentos**.

As transações estão registradas no arquivo **data.csv** (*anexo*).

Problema de Negócio

Como o e-commerce pode **identificar padrões de compra**, segmentar clientes de forma eficiente e **reduzir cancelamentos** para aumentar a lucratividade?

Objetivos

1. Compreensão do Negócio e do Problema

- Entender a dinâmica do e-commerce, os principais desafios e o que os dados podem revelar sobre os clientes.
- Definir métricas de sucesso, como taxa de cancelamento, ticket médio e recorrência de compras.

2. Compreensão dos Dados

- Explorar a estrutura do dataset fornecido.
- Verificar a qualidade dos dados: tratar valores ausentes, duplicados e inconsistentes.
- Criar novas variáveis que possam ajudar na análise.

3. Análise Exploratória de Dados (EDA)

- Investigar padrões de compras ao longo do tempo.
- Identificar os produtos mais vendidos e quais são frequentemente comprados juntos.
- Avaliar o impacto dos preços e descontos nas vendas.
- Analisar padrões de cancelamento de pedidos.

4. Segmentação de Clientes

- Criar **grupos de clientes** com base em comportamentos de compra.
- Aplicar técnicas como **RFM (Recência, Frequência e Valor Monetário)** ou **K-Means Clustering**.
- Analisar as características de cada segmento e sugerir estratégias de marketing personalizadas.

5. Geração de Insights e Recomendações

- Identificar padrões que podem ajudar na **redução de cancelamentos**.
- Sugerir estratégias de fidelização para clientes de alto valor.
- Apontar oportunidades de **cross-sell** e **up-sell**.
- Fornecer insights para otimizar promoções e campanhas de marketing.

RESOLUÇÃO 2

Baseado na resolução de Farzad Nekouei disponível no Kaggle:

<https://www.kaggle.com/code/farzadnekouei/customer-segmentation-recommendation-system/notebook>

E no Github:

https://github.com/FarzadNekouee/Retail_Customer_Segmentation_Recommendation_System

Problema:

Neste projeto, focado no setor de **varejo online**, vamos analisar um **conjunto de dados transacionais** de um varejista baseado no Reino Unido.

Este conjunto de dados documenta todas as transações entre 2010 e 2011.

Nosso objetivo principal é ampliar a eficiência das estratégias de marketing e aumentar as vendas por meio da **segmentação de clientes**.

Pretendemos transformar os dados transacionais em um conjunto de dados centrado no cliente, criando novas características que facilitarão a segmentação dos clientes em grupos distintos usando o algoritmo de **agrupamento K-means**.

Essa segmentação nos permitirá entender os **perfis** e preferências de diferentes grupos de clientes.

Com base nisso, pretendemos desenvolver, um **sistema de recomendação** que sugerirá os produtos mais vendidos para clientes dentro de cada segmento que ainda não compraram esses itens, aumentando assim a eficácia do marketing e impulsionando as vendas.

Objetivos:

- **Limpeza e Transformação de Dados:** Limpar o conjunto de dados, lidando com valores ausentes, duplicatas e outliers, preparando-o para um agrupamento eficaz.
- **Engenharia de Características:** Desenvolver novas características com base nos dados transacionais para criar um conjunto de dados centrado no cliente, estabelecendo a base para a segmentação de clientes.
- **Pré-processamento de Dados:** Realizar escalonamento de características e redução de dimensionalidade para simplificar os dados, aumentando a eficiência do processo de agrupamento.
- **Segmentação de Clientes usando Agrupamento K-Means:** Segmentar os clientes em grupos distintos usando K-means, facilitando o marketing direcionado e estratégias personalizadas.
- **Análise e Avaliação de Clusters:** Analisar e perfilar cada cluster para desenvolver estratégias de marketing direcionadas e avaliar a qualidade dos clusters formados.
- **Sistema de Recomendação:** Implementar um sistema para recomendar os produtos mais vendidos para clientes dentro do mesmo cluster que ainda não compraram esses produtos, visando aumentar as vendas e a eficácia do marketing.

Sumário:

1. Passo 1 | Configuração e Inicialização

- 1.1 Importação das Bibliotecas Necessárias
- 1.2 Carregamento do Conjunto de Dados

2. Passo 2 | Análise Inicial dos Dados

- 2.1 Visão Geral do Conjunto de Dados
- 2.2 Estatísticas Resumidas

3. Passo 3 | Limpeza e Transformação de Dados

- 3.1 Tratamento de Valores Ausentes
- 3.2 Tratamento de Duplicatas
- 3.3 Tratamento de Transações Canceladas
- 3.4 Correção de Anomalias no Código de Estoque
- 3.5 Limpeza da Coluna de Descrição
- 3.6 Tratamento de Preços Unitários Zero
- 3.7 Tratamento de Outliers

4. Passo 4 | Engenharia de Características

- 4.1 Características RFM
 - 4.1.1 Recência (R)
 - 4.1.2 Frequência (F)
 - 4.1.3 Monetário (M)
- 4.2 Diversidade de Produtos
- 4.3 Características Comportamentais
- 4.4 Características Geográficas
- 4.5 Insights sobre Cancelamentos
- 4.6 Sazonalidade e Tendências

5. Passo 5 | Detecção e Tratamento de Outliers

6. Passo 6 | Análise de Correlação

7. Passo 7 | Escalonamento de Características

8. Passo 8 | Redução de Dimensionalidade

9. Passo 9 | Agrupamento K-Means

- 9.1 Determinação do Número Ótimo de Clusters

- 9.1.1 Método do Cotovelo
- 9.1.2 Método da Silhueta
- 9.2 Modelo de Agrupamento - K-means

10. Passo 10 | Avaliação do Agrupamento

- 10.1 Visualização 3D dos Principais Componentes Principais
- 10.2 Visualização da Distribuição dos Clusters
- 10.3 Métricas de Avaliação

11. Passo 11 | Análise e Perfilamento dos Clusters

- 11.1 Abordagem do Gráfico de Radar
- 11.2 Abordagem do Gráfico de Histograma

12. Passo 12 | Sistema de Recomendação

Passo 1 | Configuração e Inicialização

Passo 1.1 | Importação das Bibliotecas Necessárias

Primeiramente, vamos importar todas as bibliotecas necessárias que usaremos ao longo do projeto. Isso geralmente inclui bibliotecas para manipulação de dados, visualização de dados e outras, dependendo das necessidades específicas do projeto:

```
# Ignorar avisos
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import plotly.graph_objects as go
from matplotlib.colors import LinearSegmentedColormap
from matplotlib import colors as mcolors
from scipy.stats import linregress
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from yellowbrick.cluster import KElbowVisualizer, SilhouetteVisualizer
from sklearn.metrics import silhouette_score, calinski_harabasz_score,
davies_bouldin_score
from sklearn.cluster import KMeans
from tabulate import tabulate
from collections import Counter
%matplotlib inline
```

Passo 1.2 | Carregamento do Conjunto de Dados

Em seguida, vamos carregar o conjunto de dados em um DataFrame do pandas, o que facilitará a manipulação e análise:

```
# Carregar o conjunto de dados
df = pd.read_csv('/kaggle/input/ecommerce-data/data.csv', encoding="ISO-8859-1")
```

Descrição do Conjunto de Dados:

Variável	Descrição
InvoiceNo	Código que representa cada transação única. Se o código começa com 'c', indica um cancelamento.
StockCode	Código atribuído exclusivamente a cada produto distinto.
Description	Descrição de cada produto.
Quantity	O número de unidades de um produto em uma transação.
InvoiceDate	A data e hora da transação.

Variável	Descrição
UnitPrice	O preço unitário do produto em libras esterlinas.
CustomerID	Identificador atribuído exclusivamente a cada cliente.
Country	O país do cliente.

Passo 2 | Análise Inicial dos Dados

Passo 2.1 | Visão Geral do Conjunto de Dados

Primeiramente, vamos realizar uma análise preliminar para entender a estrutura e os tipos de colunas de dados:

```
# Exibir as primeiras 10 linhas do conjunto de dados
df.head(10)

# Mostrar as informações do dataset
df.info()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
5	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65	17850.0	United Kingdom
6	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8:26	4.25	17850.0	United Kingdom
7	536366	22633	HAND WARMER UNION JACK	6	12/1/2010 8:28	1.85	17850.0	United Kingdom
8	536366	22632	HAND WARMER RED POLKA DOT	6	12/1/2010 8:28	1.85	17850.0	United Kingdom
9	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	12/1/2010 8:34	1.69	13047.0	United Kingdom

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        541909 non-null object
1   StockCode        541909 non-null object
2   Description      540455 non-null object
3   Quantity         541909 non-null int64
4   InvoiceDate      541909 non-null object
5   UnitPrice        541909 non-null float64
6   CustomerID       406829 non-null float64
7   Country          541909 non-null object
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
```

Inferências:

O conjunto de dados consiste em 541.909 entradas e 8 colunas. Aqui está uma breve visão geral de cada coluna:

- **InvoiceNo:** Esta é uma coluna do tipo objeto que contém o número da fatura para cada transação. Cada número de fatura pode representar vários itens comprados em uma única transação.
- **StockCode:** Uma coluna do tipo objeto que representa o código do produto para cada item.
- **Description:** Esta coluna, também do tipo objeto, contém as descrições dos produtos. Tem alguns valores ausentes, com 540.455 entradas não nulas de 541.909.

- **Quantity:** Esta é uma coluna do tipo inteiro que indica a quantidade de produtos comprados em cada transação.
- **InvoiceDate:** Uma coluna de data e hora que registra a data e hora de cada transação.
- **UnitPrice:** Uma coluna do tipo float que representa o preço unitário de cada produto.
- **CustomerID:** Uma coluna do tipo float que contém o ID do cliente para cada transação. Esta coluna tem um número significativo de valores ausentes, com apenas 406.829 entradas não nulas de 541.909.
- **Country:** Uma coluna do tipo objeto que registra o país onde cada transação ocorreu.

Passo 2.2 | Estatísticas Resumidas

Agora, vamos gerar estatísticas resumidas para obter insights iniciais sobre a distribuição dos dados:

```
# Estatísticas resumidas para variáveis numéricas  
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Quantity	541909.0	9.552250	218.081158	-80995.00	1.00	3.00	10.00	80995.0
UnitPrice	541909.0	4.611114	96.759853	-11062.06	1.25	2.08	4.13	38970.0
CustomerID	406829.0	15287.690570	1713.600303	12346.00	13953.00	15152.00	16791.00	18287.0

Inferências:

- **Quantity:** A quantidade média de produtos em uma transação é aproximadamente 9,55. A quantidade tem uma ampla variação, com um valor mínimo de -80995 e um valor máximo de 80995. Os valores negativos indicam pedidos devolvidos ou cancelados, que precisam ser tratados adequadamente.
 - **UnitPrice:** O preço unitário médio dos produtos é aproximadamente 4,61. O preço unitário também mostra uma ampla variação, de -11062,06 a 38970, o que sugere a presença de erros ou ruídos nos dados, já que preços negativos não fazem sentido.
 - **CustomerID:** Há 406.829 entradas não nulas, indicando valores ausentes no conjunto de dados que precisam ser tratados. Os IDs dos clientes variam de 12346 a 18287, ajudando a identificar clientes únicos.
-

Passo 3 | Limpeza e Transformação de Dados

Passo 3.1 | Tratamento de Valores Ausentes

Inicialmente, vamos determinar a porcentagem de valores ausentes presentes em cada coluna, seguido pela seleção da estratégia mais eficaz para tratá-los:

```
# Calcular a porcentagem de valores ausentes para cada coluna
missing_data = df.isnull().sum()
missing_percentage = (missing_data[missing_data > 0] / df.shape[0]) * 100

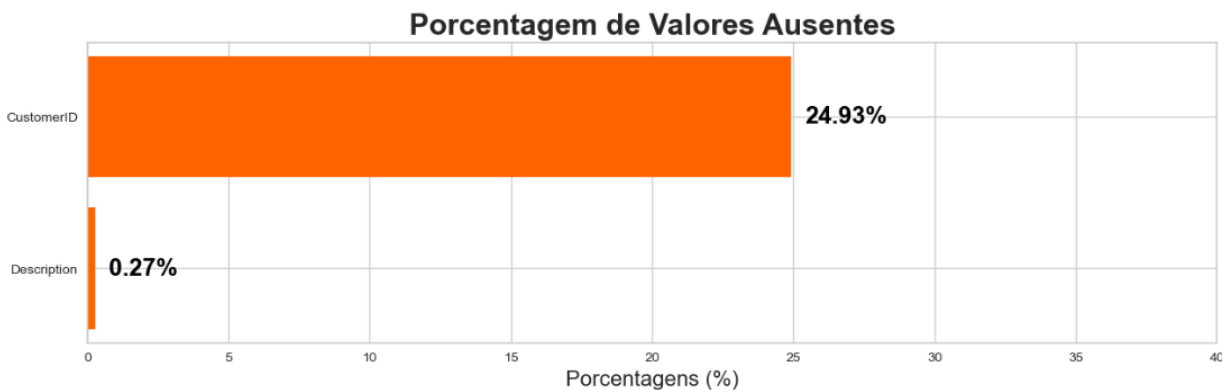
# Preparar os valores
missing_percentage.sort_values(ascending=True, inplace=True)

# Plotar o gráfico de barras horizontais
fig, ax = plt.subplots(figsize=(15, 4))
ax.barh(missing_percentage.index, missing_percentage, color='#ff6200')

# Anotar os valores e índices
for i, (value, name) in enumerate(zip(missing_percentage, missing_percentage.index)):
    ax.text(value+0.5, i, f"{value:.2f}%", ha='left', va='center', fontweight='bold',
    fontsize=18, color='black')

# Definir o limite do eixo x
ax.set_xlim([0, 40])

# Adicionar título e rótulo do eixo x
plt.title("Porcentagem de Valores Ausentes", fontweight='bold', fontsize=22)
plt.xlabel('Porcentagens (%)', fontsize=16)
plt.show()
```



Estratégia para Tratamento de Valores Ausentes:

- **CustomerID (24,93% de valores ausentes):** A coluna CustomerID contém quase um quarto de dados ausentes. Esta coluna é essencial para agrupar clientes e criar um sistema de recomendação. Imputar uma porcentagem tão grande de valores ausentes pode introduzir viés ou ruído significativo na análise. Portanto, a remoção das linhas com CustomerID ausente parece ser a abordagem mais razoável para manter a integridade dos clusters e da análise.
- **Description (0,27% de valores ausentes):** A coluna Description tem uma pequena porcentagem de valores ausentes. No entanto, foi observado que há inconsistências nos dados onde o mesmo StockCode nem sempre tem a mesma Description. Isso indica problemas de qualidade dos dados e possíveis erros nas descrições dos produtos. Dada a baixa porcentagem de valores ausentes, seria prudente remover as linhas

com Description ausente para evitar propagar erros e inconsistências nas análises subsequentes.

Passo 3.2 | Tratamento de Duplicatas

Em seguida, vamos identificar linhas duplicadas no conjunto de dados:

```
# Encontrar linhas duplicadas (mantendo todas as instâncias)
duplicate_rows = df[df.duplicated(keep=False)]

# Ordenar os dados por determinadas colunas para ver as linhas duplicadas lado a lado
duplicate_rows_sorted = duplicate_rows.sort_values(by=['InvoiceNo', 'StockCode',
'Description', 'CustomerID', 'Quantity'])

# Exibir os primeiros 10 registros
duplicate_rows_sorted.head(10)
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
494	536409	21866	UNION JACK FLAG LUGGAGE TAG	1	12/1/2010 11:45	1.25	17908.0	United Kingdom
517	536409	21866	UNION JACK FLAG LUGGAGE TAG	1	12/1/2010 11:45	1.25	17908.0	United Kingdom
485	536409	22111	SCOTTIE DOG HOT WATER BOTTLE	1	12/1/2010 11:45	4.95	17908.0	United Kingdom
539	536409	22111	SCOTTIE DOG HOT WATER BOTTLE	1	12/1/2010 11:45	4.95	17908.0	United Kingdom
489	536409	22866	HAND WARMER SCOTTIE DOG DESIGN	1	12/1/2010 11:45	2.10	17908.0	United Kingdom
527	536409	22866	HAND WARMER SCOTTIE DOG DESIGN	1	12/1/2010 11:45	2.10	17908.0	United Kingdom
521	536409	22900	SET 2 TEA TOWELS I LOVE LONDON	1	12/1/2010 11:45	2.95	17908.0	United Kingdom
537	536409	22900	SET 2 TEA TOWELS I LOVE LONDON	1	12/1/2010 11:45	2.95	17908.0	United Kingdom
578	536412	21448	12 DAISY PEGS IN WOOD BOX	1	12/1/2010 11:49	1.65	17920.0	United Kingdom
598	536412	21448	12 DAISY PEGS IN WOOD BOX	1	12/1/2010 11:49	1.65	17920.0	United Kingdom

Estratégia para Tratamento de Duplicatas:

No contexto deste projeto, a presença de linhas completamente idênticas, incluindo horários de transação idênticos, sugere que esses podem ser erros de registro de dados em vez de transações genuinamente repetidas. Manter essas linhas duplicadas pode introduzir ruído e potenciais imprecisões no agrupamento e no sistema de recomendação. Portanto, vamos remover essas linhas duplicadas completamente idênticas do conjunto de dados.

```
# Exibir o número de linhas duplicadas
print(f"O conjunto de dados contém {df.duplicated().sum()} linhas duplicadas que precisam ser removidas.")

# Remover linhas duplicadas
df.drop_duplicates(inplace=True)

# Verificar o número de linhas no dataframe
df.shape[0]
```

O conjunto de dados contém 5268 linhas duplicadas que precisam ser removidas.

536641

Passo 4 | Engenharia de Características

Passo 4.1 | Características RFM

RFM é um método usado para analisar o valor do cliente e segmentar a base de clientes. É um acrônimo que significa:

- **Recência (R):** Esta métrica indica o quão recentemente um cliente fez uma compra. Um valor de recência mais baixo significa que o cliente comprou mais recentemente, indicando maior engajamento com a marca.
- **Frequência (F):** Esta métrica indica a frequência com que um cliente faz uma compra dentro de um determinado período. Um valor de frequência mais alto indica um cliente que interage mais frequentemente com o negócio, sugerindo maior fidelidade ou satisfação.
- **Monetário (M):** Esta métrica representa o valor total gasto por um cliente em um determinado período. Clientes com um valor monetário mais alto contribuíram mais para o negócio, indicando seu potencial alto valor de vida.

Juntas, essas métricas ajudam a entender o comportamento de compra e as preferências dos clientes, o que é fundamental para personalizar estratégias de marketing e criar um sistema de recomendação.

Passo 4.1.1 | Recência (R)

Neste passo, vamos focar em entender o quão recentemente um cliente fez uma compra. Este é um aspecto crucial da segmentação de clientes, pois ajuda a identificar o nível de engajamento dos clientes. Aqui, vou definir a seguinte característica:

- **Dias Desde a Última Compra:** Esta característica representa o número de dias que se passaram desde a última compra do cliente. Um valor mais baixo indica que o cliente comprou recentemente, implicando um maior nível de engajamento com o negócio, enquanto um valor mais alto pode indicar um lapso ou diminuição do engajamento.

```
# Converter InvoiceDate para o tipo datetime
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

# Converter InvoiceDate para datetime e extrair apenas a data
df['InvoiceDay'] = df['InvoiceDate'].dt.date

# Encontrar a data de compra mais recente para cada cliente
customer_data = df.groupby('CustomerID')['InvoiceDay'].max().reset_index()

# Encontrar a data mais recente em todo o conjunto de dados
most_recent_date = df['InvoiceDay'].max()

# Converter InvoiceDay para o tipo datetime antes da subtração
customer_data['InvoiceDay'] = pd.to_datetime(customer_data['InvoiceDay'])
most_recent_date = pd.to_datetime(most_recent_date)

# Calcular o número de dias desde a última compra para cada cliente
customer_data['Days_Since_Last_Purchase'] = (most_recent_date -
customer_data['InvoiceDay']).dt.days
```

```
# Remover a coluna InvoiceDay
customer_data.drop(columns=['InvoiceDay'], inplace=True)

# Exibir as primeiras linhas do dataframe customer_data
customer_data.head()
```

	CustomerID	Days_Since_Last_Purchase
0	12346.0	325
1	12347.0	2
2	12348.0	75
3	12349.0	18
4	12350.0	310

Passo 4.1.2 | Frequência (F)

Neste passo, vou criar duas características que quantificam a frequência de engajamento de um cliente com o varejista:

- **Total de Transações:** Esta característica representa o número total de transações feitas por um cliente. Ajuda a entender o nível de engajamento de um cliente com o varejista.
- **Total de Produtos Comprados:** Esta característica indica o número total de produtos (soma das quantidades) comprados por um cliente em todas as transações. Dá uma visão do comportamento de compra do cliente em termos do volume de produtos comprados.

```
# Calcular o número total de transações feitas por cada cliente
total_transactions = df.groupby('CustomerID')['InvoiceNo'].nunique().reset_index()
total_transactions.rename(columns={'InvoiceNo': 'Total_Transactions'}, inplace=True)

# Calcular o número total de produtos comprados por cada cliente
total_products_purchased = df.groupby('CustomerID')['Quantity'].sum().reset_index()
total_products_purchased.rename(columns={'Quantity': 'Total_Products_Purchased'},
inplace=True)

# Mesclar as novas características no dataframe customer_data
customer_data = pd.merge(customer_data, total_transactions, on='CustomerID')
customer_data = pd.merge(customer_data, total_products_purchased, on='CustomerID')

# Exibir as primeiras linhas do dataframe customer_data
customer_data.head()
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased
0	12346.0	325	2	0
1	12347.0	2	7	2458
2	12348.0	75	4	2341
3	12349.0	18	1	631
4	12350.0	310	1	197

Passo 4.1.3 | Monetário (M)

Neste passo, vou criar duas características que representam o aspecto monetário das transações dos clientes:

- **Gasto Total:** Esta característica representa o valor total gasto por cada cliente. É calculado como a soma do produto de UnitPrice e Quantity para todas as transações feitas por um cliente.
- **Valor Médio da Transação:** Esta característica é calculada como o **Gasto Total** dividido pelo **Total de Transações** para cada cliente. Indica o valor médio de uma transação realizada por um cliente.

```
# Calcular o gasto total por cada cliente
df['Total_Spend'] = df['UnitPrice'] * df['Quantity']
total_spend = df.groupby('CustomerID')['Total_Spend'].sum().reset_index()

# Calcular o valor médio da transação para cada cliente
average_transaction_value = total_spend.merge(total_transactions, on='CustomerID')
average_transaction_value['Average_Transaction_Value'] =
average_transaction_value['Total_Spend'] /
average_transaction_value['Total_Transactions']

# Mesclar as novas características no dataframe customer_data
customer_data = pd.merge(customer_data, total_spend, on='CustomerID')
customer_data = pd.merge(customer_data, average_transaction_value[['CustomerID',
'Average_Transaction_Value']], on='CustomerID')

# Exibir as primeiras linhas do dataframe customer_data
customer_data.head()
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_Transaction_Value
0	12346.0	325	2	0	0.00	0.000000
1	12347.0	2	7	2458	4310.00	615.714286
2	12348.0	75	4	2341	1797.24	449.310000
3	12349.0	18	1	631	1757.55	1757.550000
4	12350.0	310	1	197	334.40	334.400000

Passo 5 | Detecção e Tratamento de Outliers

Nesta seção, vamos identificar e tratar outliers no nosso conjunto de dados. Outliers são pontos de dados que são significativamente diferentes da maioria dos outros pontos no conjunto de dados. Esses pontos podem potencialmente distorcer os resultados da nossa análise, especialmente no agrupamento K-means, onde podem influenciar significativamente a posição dos centróides dos clusters. Portanto, é essencial identificar e tratar esses outliers adequadamente para obter resultados de agrupamento mais precisos e significativos.

Dada a natureza multidimensional dos dados, seria prudente usar algoritmos que possam detectar outliers em espaços multidimensionais.

Vamos usar o algoritmo **Isolation Forest** para essa tarefa. Este algoritmo funciona bem para dados multidimensionais e é computacionalmente eficiente. Ele isola observações selecionando aleatoriamente uma característica e, em seguida, selecionando aleatoriamente um valor de divisão entre os valores máximo e mínimo da característica selecionada.

```
# Inicializar o modelo Isolation Forest com um parâmetro de contaminação de 0,05
model = IsolationForest(contamination=0.05, random_state=0)

# Ajustar o modelo ao nosso conjunto de dados (convertendo DataFrame para NumPy para evitar avisos)
customer_data['Outlier_Scores'] = model.fit_predict(customer_data.iloc[:, 1:].to_numpy())

# Criar uma nova coluna para identificar outliers (1 para inliers e -1 para outliers)
customer_data['Is_Outlier'] = [1 if x == -1 else 0 for x in customer_data['Outlier_Scores']]

# Exibir as primeiras linhas do dataframe customer_data
customer_data.head()
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_Transaction_Value	Outlier_Scores	Is_Outlier
0	12346.0	325	2	0	0.00	0.000000	1	0
1	12347.0	2	7	2458	4310.00	615.714286	1	0
2	12348.0	75	4	2341	1797.24	449.310000	1	0
3	12349.0	18	1	631	1757.55	1757.550000	-1	1
4	12350.0	310	1	197	334.40	334.400000	1	0

Estratégia para Tratamento de Outliers:

Considerando a natureza do projeto (segmentação de clientes usando agrupamento), é crucial lidar com esses outliers para evitar que eles afetem significativamente a qualidade dos clusters. Portanto, vamos separar esses outliers para análise posterior e removê-los do nosso conjunto de dados principal para prepará-lo para a análise de agrupamento.

```
# Separar os outliers para análise
outliers_data = customer_data[customer_data['Is_Outlier'] == 1]

# Remover os outliers do conjunto de dados principal
customer_data_cleaned = customer_data[customer_data['Is_Outlier'] == 0]
```

```
# Remover as colunas 'Outlier_Scores' e 'Is_Outlier'
customer_data_cleaned = customer_data_cleaned.drop(columns=['Outlier_Scores',
'Is_Outlier'])

# Redefinir o índice dos dados limpos
customer_data_cleaned.reset_index(drop=True, inplace=True)

# Verificar o número de linhas no conjunto de dados limpo
customer_data_cleaned.shape[0]
```

4153

Passo 6 | Análise de Correlação

Antes de prosseguir com o agrupamento K-means, é essencial verificar a correlação entre as características no nosso conjunto de dados. A presença de **multicolinearidade**, onde **as características são altamente correlacionadas**, pode potencialmente afetar o processo de agrupamento, impedindo que o modelo aprenda os padrões subjacentes reais nos dados, já que as características não fornecem informações únicas. Isso pode levar a clusters que não são bem separados e significativos.

Se identificarmos multicolinearidade, podemos utilizar técnicas de redução de dimensionalidade como PCA. Essas técnicas ajudam a neutralizar o efeito da multicolinearidade, transformando as características correlacionadas em um novo conjunto de variáveis não correlacionadas, preservando a maior parte da variância dos dados originais. Essa etapa não apenas melhora a qualidade dos clusters formados, mas também torna o processo de agrupamento mais eficiente computacionalmente.

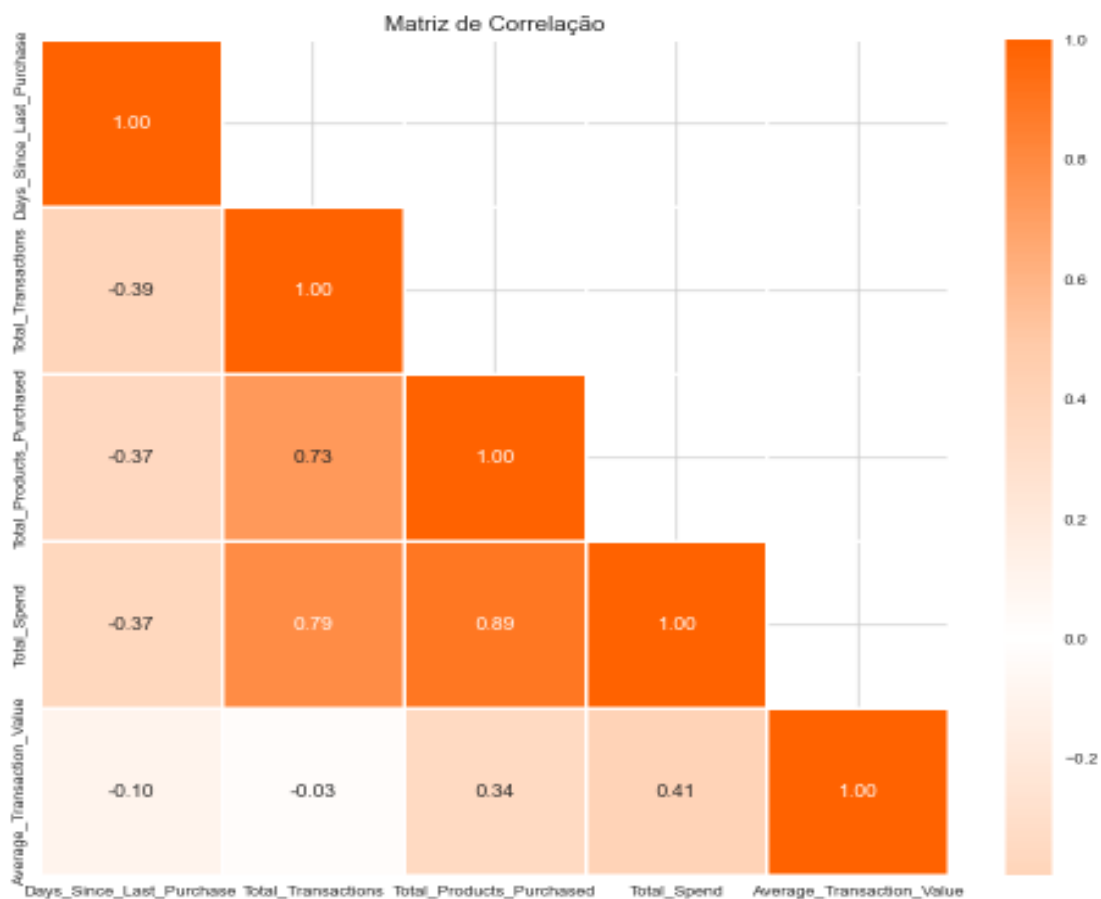
```
# Redefinir o estilo de fundo
sns.set_style('whitegrid')

# Calcular a matriz de correlação, excluindo a coluna 'CustomerID'
corr = customer_data_cleaned.drop(columns=['CustomerID']).corr()

# Definir um mapa de cores personalizado
colors = ['#ff6200', '#ffcaa8', 'white', '#ffcaa8', '#ff6200']
my_cmap = LinearSegmentedColormap.from_list('custom_map', colors, N=256)

# Criar uma máscara para mostrar apenas o triângulo inferior da matriz (já que é
# espelhado em torno de sua diagonal)
mask = np.zeros_like(corr)
mask[np.triu_indices_from(mask, k=1)] = True

# Plotar o mapa de calor
plt.figure(figsize=(12, 10))
sns.heatmap(corr, mask=mask, cmap=my_cmap, annot=True, center=0, fmt='.2f', linewidths=2)
plt.title('Matriz de Correlação', fontsize=14)
plt.show()
```



Inferências:

Observando o mapa de calor, podemos ver que há alguns pares de variáveis que têm altas correlações, por exemplo:

- Total_Spend e Total_Products_Purchased
- Total_Transactions e Total_Spend
- Total_Transactions e Total_Products_Purchased

Essas altas correlações indicam que essas variáveis se movem juntas, implicando um grau de multicolinearidade.

Passo 7 | Escalonamento de Características

Antes de prosseguir com o agrupamento e a redução de dimensionalidade, é imperativo escalonar nossas características. Esta etapa é de grande importância, especialmente no contexto de algoritmos baseados em distância, como K-means, e métodos de redução de dimensionalidade, como PCA. Aqui está o porquê:

- **Para Agrupamento K-means:** K-means depende fortemente do conceito de 'distância' entre pontos de dados para formar clusters. Quando as características não estão em uma escala semelhante, características com valores maiores podem influenciar desproporcionalmente o resultado do agrupamento, potencialmente levando a agrupamentos incorretos.
- **Para PCA:** O PCA visa encontrar as direções onde os dados variam mais. Quando as características não são escalonadas, aquelas com valores maiores podem dominar esses componentes, não refletindo com precisão os padrões subjacentes nos dados.

Metodologia:

Portanto, para garantir uma influência equilibrada no modelo e revelar os verdadeiros padrões nos dados, vamos padronizar nossos dados, transformando as características para ter uma média de 0 e um desvio padrão de 1. No entanto, nem todas as características precisam ser escalonadas. Aqui estão as exceções e os motivos pelos quais elas são excluídas:

- **CustomerID:** Esta característica é apenas um identificador para os clientes e não contém informações significativas para o agrupamento.
- **Is_UK:** Esta é uma característica binária que indica se o cliente é do Reino Unido ou não. Como já assume valores de 0 ou 1, escaloná-la não fará diferença significativa.
- **Day_Of_Week:** Esta característica representa o dia da semana mais frequente em que o cliente fez transações. Como é uma característica categórica representada por inteiros (1 a 7), escaloná-la não seria necessário.

Vamos prosseguir para escalonar as outras características no conjunto de dados para prepará-lo para PCA e agrupamento K-means.

```
# Inicializar o StandardScaler
scaler = StandardScaler()

# Lista de colunas que não precisam ser escalonadas
columns_to_exclude = ['CustomerID', 'Is_UK', 'Day_Of_Week']

# Lista de colunas que precisam ser escalonadas
columns_to_scale = customer_data_cleaned.columns.difference(columns_to_exclude)

# Copiar o conjunto de dados limpo
customer_data_scaled = customer_data_cleaned.copy()

# Aplicar o escalonador às colunas necessárias no conjunto de dados
customer_data_scaled[columns_to_scale] =
scaler.fit_transform(customer_data_scaled[columns_to_scale])

# Exibir as primeiras linhas dos dados escalonados
customer_data_scaled.head()
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_Transaction_Value
0	12346.0	2.295770	-0.497009	-0.818184	-0.853863	-1.294572
1	12347.0	-0.909848	0.777113	2.308159	2.570218	1.572465
2	12348.0	-0.185359	0.012640	2.159346	0.573955	0.797613
3	12350.0	2.146902	-0.751833	-0.567619	-0.588198	0.262541
4	12352.0	-0.572415	1.796411	-0.220388	0.373889	-0.640381

Passo 8 | Redução da Dimensionalidade

Por que Precisamos de Redução da Dimensionalidade?

A redução da dimensionalidade é uma etapa crucial para simplificar os dados, especialmente quando lidamos com muitas características. Aqui estão os principais motivos para aplicar essa técnica:

1. **Multicolinearidade Detectada:** Identificamos que algumas características no nosso conjunto de dados estão altamente correlacionadas. A redução da dimensionalidade ajuda a remover essa redundância.
2. **Melhor Agrupamento com K-means:** O algoritmo K-means é sensível ao número de características. Reduzir a dimensionalidade ajuda a melhorar a qualidade dos clusters, tornando-os mais compactos e bem separados.
3. **Redução de Ruído:** Ao focar nas características mais importantes, podemos eliminar ruídos nos dados, resultando em clusters mais precisos.
4. **Visualização Aprimorada:** Reduzir os dados para 2 ou 3 dimensões facilita a visualização dos clusters, permitindo uma análise mais intuitiva.
5. **Eficiência Computacional:** Menos características significam menos cálculos, o que torna o processo de agrupamento mais rápido e eficiente.

Qual Método de Redução da Dimensionalidade?

Neste projeto, optamos por usar o **PCA (Análise de Componentes Principais)**. O PCA é uma técnica linear que transforma os dados em um novo conjunto de características não correlacionadas, chamadas de componentes principais. Esses componentes capturam a maior variância nos dados, permitindo que reduzamos a dimensionalidade sem perder muita informação.

Metodologia:

Vou aplicar o PCA em todos os componentes disponíveis e plotar a variância cumulativa explicada por eles. Isso nos ajudará a determinar quantos componentes principais devemos reter para capturar a maior parte da variância nos dados.

```
# Configurar CustomerID como a coluna de índice
customer_data_scaled.set_index('CustomerID', inplace=True)

# Aplicar PCA
pca = PCA().fit(customer_data_scaled)

# Calcular a variância cumulativa explicada
explained_variance_ratio = pca.explained_variance_ratio_
cumulative_explained_variance = np.cumsum(explained_variance_ratio)

# Definir o número ideal de componentes (k) com base na análise
optimal_k = 6

# Configurar o estilo do gráfico
sns.set(rc={'axes.facecolor': '#fcf0dc'}, style='darkgrid')

# Plotar a variância cumulativa explicada
plt.figure(figsize=(20, 10))
```

```

barplot = sns.barplot(x=list(range(1, len(cumulative_explained_variance) + 1)),
                      y=explained_variance_ratio,
                      color='#fcc36d',
                      alpha=0.8)

# Gráfico de linha para a variância cumulativa
lineplot, = plt.plot(range(0, len(cumulative_explained_variance)),
                     cumulative_explained_variance,
                     marker='o', linestyle='--', color='#ff6200', linewidth=2)

# Linha indicando o número ideal de componentes
optimal_k_line = plt.axvline(optimal_k - 1, color='red', linestyle='--', label=f'Valor  
ótimo de k = {optimal_k}')

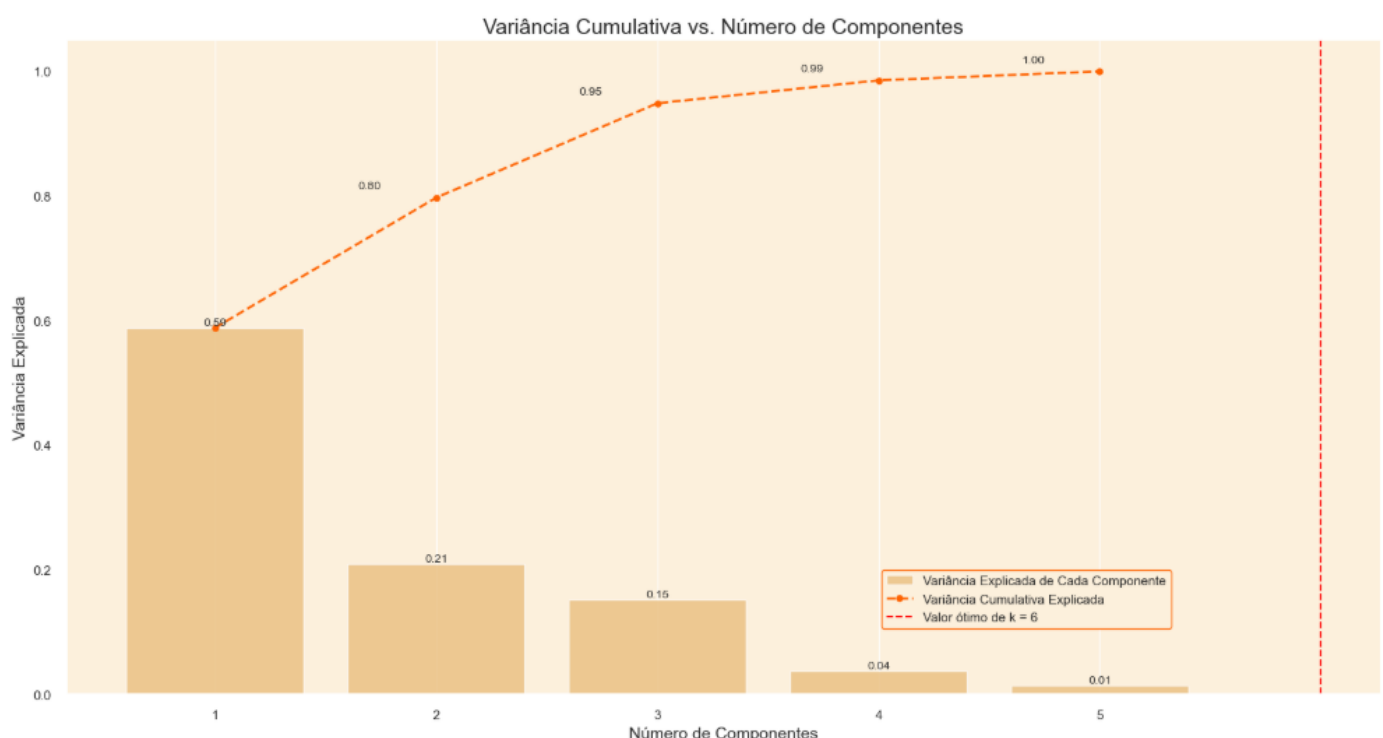
# Configurar rótulos e título
plt.xlabel('Número de Componentes', fontsize=14)
plt.ylabel('Variância Explicada', fontsize=14)
plt.title('Variância Cumulativa vs. Número de Componentes', fontsize=18)

# Adicionar legenda
plt.legend(handles=[barplot.patches[0], lineplot, optimal_k_line],
           labels=['Variância Explicada de Cada Componente', 'Variância Cumulativa  
Explicada', f'Valor ótimo de k = {optimal_k}'],
           loc=(0.62, 0.1),
           frameon=True,
           framealpha=1.0,
           edgecolor='#ff6200')

# Exibir os valores de variância no gráfico
x_offset = -0.3
y_offset = 0.01
for i, (ev_ratio, cum_ev_ratio) in enumerate(zip(explained_variance_ratio,
                                                cumulative_explained_variance)):
    plt.text(i, ev_ratio, f"{ev_ratio:.2f}", ha="center", va="bottom", fontsize=10)
    if i > 0:
        plt.text(i + x_offset, cum_ev_ratio + y_offset, f"{cum_ev_ratio:.2f}",
                 ha="center", va="bottom", fontsize=10)

plt.grid(axis='both')
plt.show()

```



Conclusão:

O gráfico mostra que os primeiros 5 componentes principais capturam cerca de 81% da variância total nos dados. Portanto, decidimos reter esses 5 componentes para a próxima etapa de agrupamento.

```
# Criar um objeto PCA com 5 componentes
pca = PCA(n_components=5)

# Ajustar e transformar os dados originais
customer_data_pca = pca.fit_transform(customer_data_scaled)

# Criar um novo dataframe com os componentes principais
customer_data_pca = pd.DataFrame(customer_data_pca,
                                  columns=['PC' + str(i + 1) for i in
range(pca.n_components_)])

# Adicionar o índice CustomerID de volta ao dataframe
customer_data_pca.index = customer_data_scaled.index

# Exibir as primeiras linhas do dataframe PCA
customer_data_pca.head()
```

	PC1	PC2	PC3	PC4	PC5
CustomerID					
12346.0	-2.190013	0.469355	1.888677	0.065634	-0.064340
12347.0	3.710552	-1.192853	0.009728	0.599719	-0.535856
12348.0	1.733176	-0.815131	0.215417	1.289200	0.534906
12350.0	-1.635207	-0.995410	1.495532	-0.075096	0.067368
12352.0	1.014304	1.390021	0.071240	-1.073338	0.187353

Passo 9 | Agrupamento K-Means

O que é K-Means?

O K-means é um algoritmo de agrupamento não supervisionado que divide os dados em um número especificado de clusters (k). Ele funciona minimizando a **soma dos quadrados dentro dos clusters (WCSS)**, também conhecida como **inércia**. O algoritmo atribui cada ponto de dados ao centróide mais próximo e atualiza os centróides iterativamente até que os clusters se estabilizem.

Desvantagens do K-Means:

1. **Sensibilidade à Inicialização:** O K-means pode convergir para um mínimo local dependendo da inicialização dos centróides.
 - **Solução:** Usar o método de inicialização **k-means++**, que escolhe os centróides iniciais de forma mais inteligente.
2. **Número de Clusters Pré-definido:** O K-means requer que o número de clusters (k) seja especificado antecipadamente.
 - **Solução:** Usar métodos como o **Método do Cotovelo** e a **Análise da Silhueta** para determinar o número ideal de clusters.
3. **Sensibilidade a Outliers:** O K-means pode ser afetado por outliers, que podem distorcer a posição dos centróides.
 - **Solução:** Remover ou tratar outliers antes de aplicar o K-means.

Passo 9.1 | Determinação do Número Ótimo de Clusters

Para determinar o número ideal de clusters (k), utilizamos dois métodos:

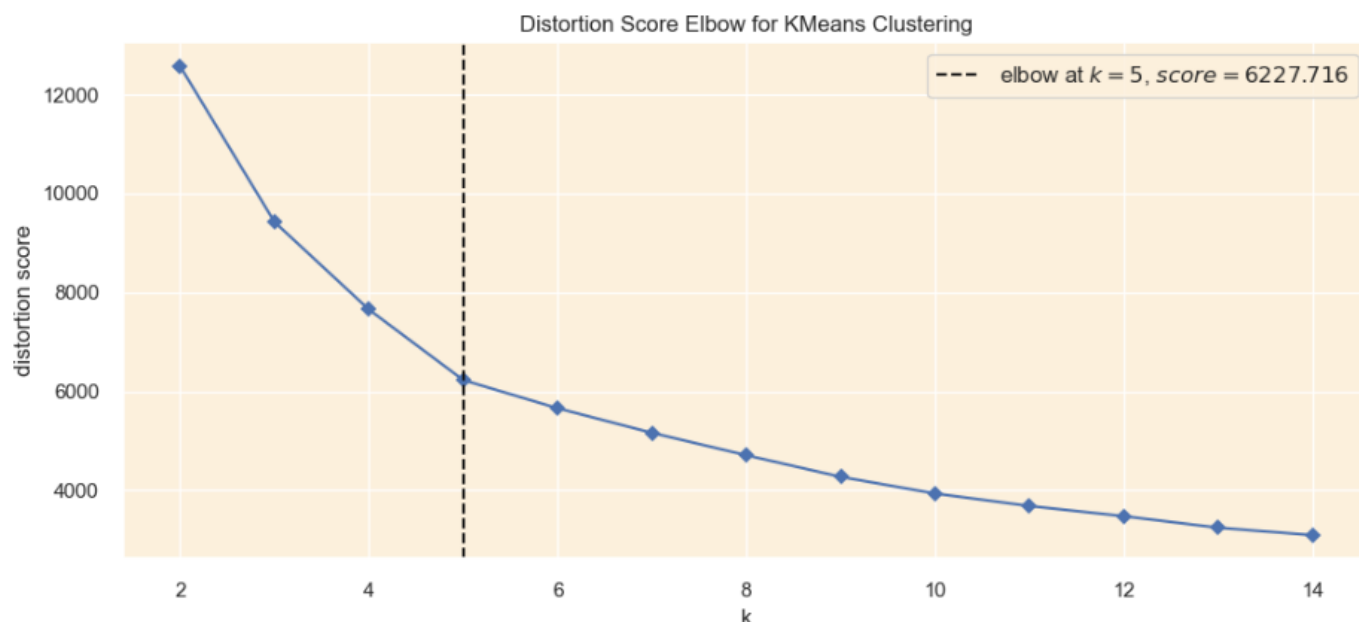
1. **Método do Cotovelo:** Avalia a inércia (WCSS) para diferentes valores de k e identifica o ponto onde a redução na inércia começa a diminuir.
2. **Método da Silhueta:** Avalia a qualidade dos clusters com base na coesão e separação, fornecendo uma pontuação que varia de -1 a 1.

Passo 9.1.1 | Método do Cotovelo

```
# Configurar o estilo do gráfico
sns.set(style='darkgrid', rc={'axes.facecolor': '#fcf0dc'})

# Instanciar o modelo K-means
km = KMeans(init='k-means++', n_init=10, max_iter=100, random_state=0)

# Criar o gráfico do Método do Cotovelo
fig, ax = plt.subplots(figsize=(12, 5))
visualizer = KElbowVisualizer(km, k=(2, 15), timings=False, ax=ax)
visualizer.fit(customer_data_pca)
visualizer.show()
```

Resultado:

O gráfico sugere que o valor ideal de k é 5. No entanto, como não há um "cotovelo" muito claro, também consideramos a **Análise da Silhueta** para confirmar.

Passo 9.1.2 | Método da Silhueta

```
def silhouette_analysis(df, start_k, stop_k, figsize=(15, 16)):
    """
    Realizar análise de silhueta para um intervalo de valores de k.
    """
    plt.figure(figsize=figsize)
    grid = gridspec.GridSpec(stop_k - start_k + 1, 2)
    first_plot = plt.subplot(grid[0, :])

    # Calcular as pontuações de silhueta
    silhouette_scores = []
    for k in range(start_k, stop_k + 1):
        km = KMeans(n_clusters=k, init='k-means++', n_init=10, max_iter=100,
random_state=0)
        km.fit(df)
        labels = km.predict(df)
        score = silhouette_score(df, labels)
        silhouette_scores.append(score)

    best_k = start_k + silhouette_scores.index(max(silhouette_scores))

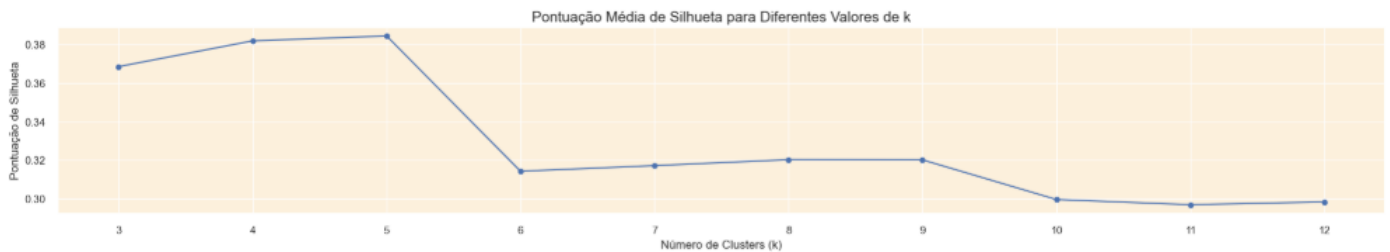
    # Plotar as pontuações de silhueta
    plt.plot(range(start_k, stop_k + 1), silhouette_scores, marker='o')
    plt.xticks(range(start_k, stop_k + 1))
    plt.xlabel('Número de Clusters (k)')
    plt.ylabel('Pontuação de Silhueta')
    plt.title('Pontuação Média de Silhueta para Diferentes Valores de k', fontsize=15)

    # Adicionar texto com o valor ótimo de k
    optimal_k_text = f'O valor de k com a maior pontuação de silhueta é: {best_k}'
    plt.text(10, 0.23, optimal_k_text, fontsize=12, bbox=dict(facecolor='#fcc36d',
edgecolor='#ff6200'))
```

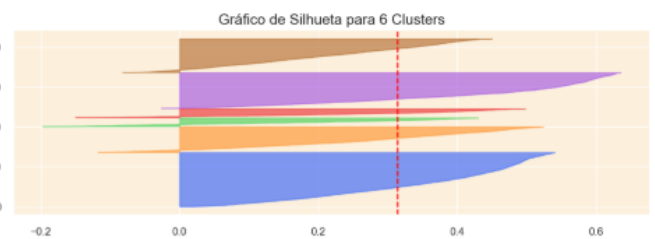
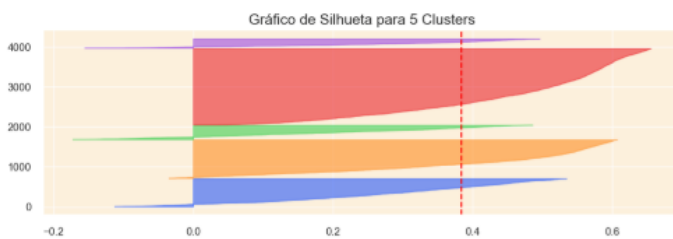
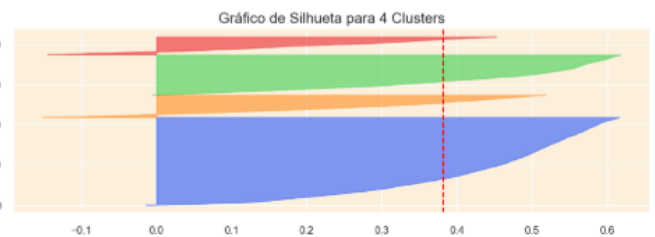
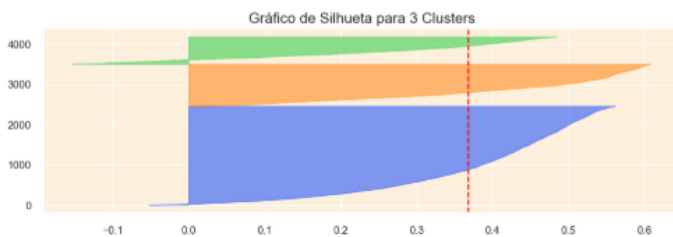
```
# Plotar gráficos de silhueta para cada k
colors = sns.color_palette("bright")
for i in range(start_k, stop_k + 1):
    km = KMeans(n_clusters=i, init='k-means++', n_init=10, max_iter=100,
random_state=0)
    row_idx, col_idx = divmod(i - start_k, 2)
    ax = plt.subplot(grid[row_idx + 1, col_idx])
    visualizer = SilhouetteVisualizer(km, colors=colors, ax=ax)
    visualizer.fit(df)
    ax.set_title(f'Gráfico de Silhueta para {i} Clusters', fontsize=15)

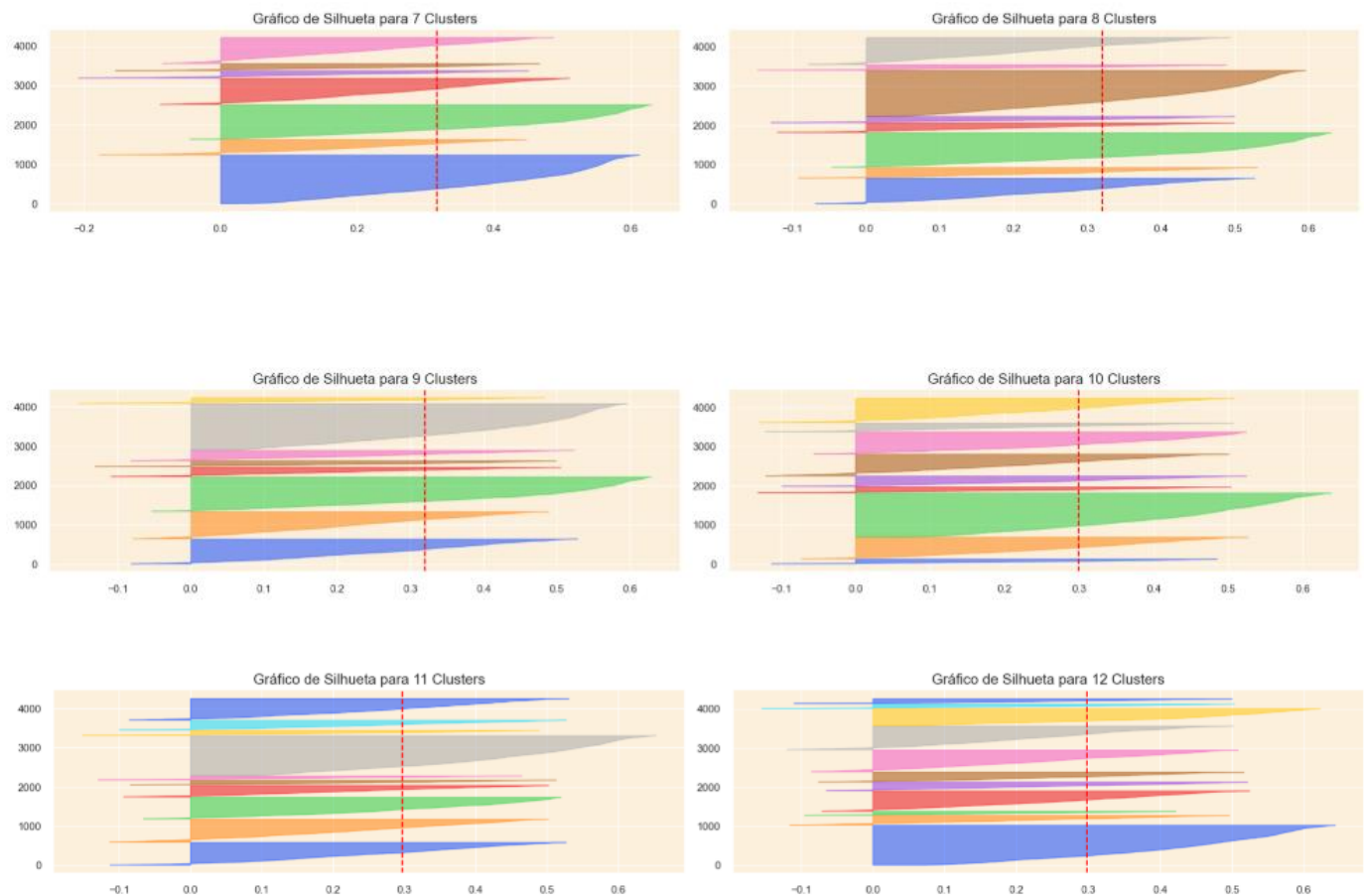
plt.tight_layout()
plt.show()

# Executar a análise de silhueta para k de 3 a 12
silhouette_analysis(customer_data_pca, 3, 12, figsize=(20, 50))
```



O valor de k com a maior pontuação de silhueta é: 5





Resultado:

A análise da silhueta sugere que **k = 3** é o valor ideal, pois fornece a maior pontuação de silhueta e clusters bem definidos.

Passo 9.2 | Aplicação do K-means

Com o número ideal de clusters definido ($k = 3$), aplicamos o algoritmo K-means:

```
# Aplicar o K-means com k = 3
kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, max_iter=100, random_state=0)
kmeans.fit(customer_data_pca)

# Adicionar os rótulos dos clusters ao dataframe original
customer_data_cleaned['cluster'] = kmeans.labels_
customer_data_pca['cluster'] = kmeans.labels_

# Exibir as primeiras linhas do dataframe com os clusters
customer_data_cleaned.head()
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_Transaction_Value	cluster
0	12346.0	325	2	0	0.00	0.000000	1
1	12347.0	2	7	2458	4310.00	615.714286	2
2	12348.0	75	4	2341	1797.24	449.310000	2
3	12350.0	310	1	197	334.40	334.400000	1
4	12352.0	36	11	470	1545.41	140.491818	0

Passo 10 | Avaliação do Agrupamento

Após determinar o número ideal de clusters (que é 3 no nosso caso) usando as análises do cotovelo e da silhueta, passamos para a etapa de avaliação para avaliar a qualidade dos clusters formados. Esta etapa é essencial para validar a eficácia do agrupamento e garantir que os clusters sejam **coerentes** e **bem separados**. As métricas de avaliação, em suma, trata-se de uma técnica de visualização, a qual vamos utilizar, e estão descritas a seguir:

1. **Visualização 3D dos Principais Componentes Principais**
2. **Visualização da Distribuição dos Clusters**
3. **Métricas de Avaliação**
 - Pontuação de Silhueta
 - Pontuação de Calinski Harabasz
 - Pontuação de Davies Bouldin

Nota: Estamos usando a versão PCA do conjunto de dados para avaliação porque é nesse espaço que os clusters foram realmente formados, capturando os padrões mais significativos nos dados. Avaliar nesse espaço garante uma representação mais precisa da qualidade do cluster, ajudando-nos a entender a verdadeira coesão e separação alcançada durante o agrupamento. Essa abordagem também ajuda a criar uma visualização 3D mais clara usando os principais componentes principais, ilustrando a separação real entre os clusters.

Passo 10.1 | Visualização 3D dos Principais Componentes Principais

Nesta parte, vou escolher os 3 principais PCs (que capturam a maior variância nos dados) e usá-los para criar uma visualização 3D. Isso nos permitirá inspecionar visualmente a qualidade da separação e coesão dos clusters até certo ponto:

```
# Configurar o esquema de cores para os clusters (ordem RGB)
colors = ['#e8000b', '#1ac938', '#023eff']

# Criar dataframes separados para cada cluster
cluster_0 = customer_data_pca[customer_data_pca['cluster'] == 0]
cluster_1 = customer_data_pca[customer_data_pca['cluster'] == 1]
cluster_2 = customer_data_pca[customer_data_pca['cluster'] == 2]

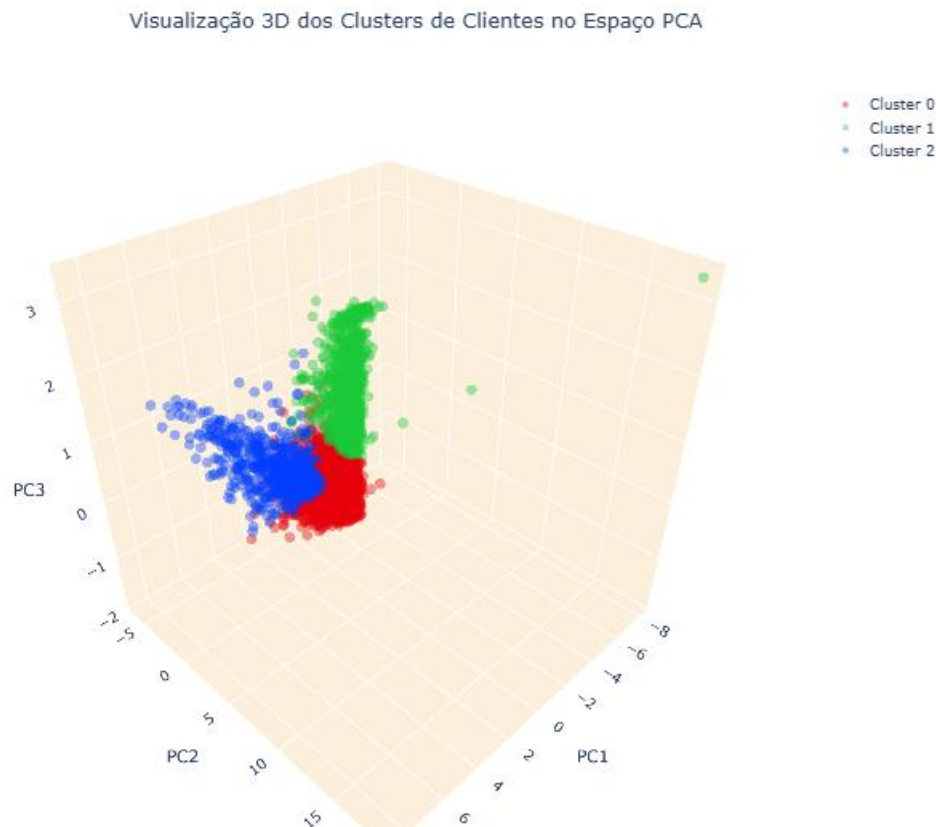
# Criar um gráfico de dispersão 3D
fig = go.Figure()

# Adicionar pontos de dados para cada cluster separadamente e especificar a cor
fig.add_trace(go.Scatter3d(x=cluster_0['PC1'], y=cluster_0['PC2'], z=cluster_0['PC3'],
                           mode='markers', marker=dict(color=colors[0], size=5, opacity=0.4),
                           name='Cluster 0'))
fig.add_trace(go.Scatter3d(x=cluster_1['PC1'], y=cluster_1['PC2'], z=cluster_1['PC3'],
                           mode='markers', marker=dict(color=colors[1], size=5, opacity=0.4),
                           name='Cluster 1'))
fig.add_trace(go.Scatter3d(x=cluster_2['PC1'], y=cluster_2['PC2'], z=cluster_2['PC3'],
                           mode='markers', marker=dict(color=colors[2], size=5, opacity=0.4),
                           name='Cluster 2'))

# Definir o título e os detalhes do layout
```

```
fig.update_layout(
    title=dict(text='Visualização 3D dos Clusters de Clientes no Espaço PCA', x=0.5),
    scene=dict(
        xaxis=dict(backgroundcolor="#fcf0dc", gridcolor='white', title='PC1'),
        yaxis=dict(backgroundcolor="#fcf0dc", gridcolor='white', title='PC2'),
        zaxis=dict(backgroundcolor="#fcf0dc", gridcolor='white', title='PC3'),
    ),
    width=900,
    height=800
)

# Mostrar o gráfico
fig.show()
```



Passo 10.2 | Visualização da Distribuição dos Clusters

Vamos utilizar um gráfico de barras para visualizar a porcentagem de clientes em cada cluster, o que ajuda a entender se os clusters estão equilibrados e significativos:

```
# Calcular a porcentagem de clientes em cada cluster
cluster_percentage = (customer_data_pca['cluster'].value_counts(normalize=True) *
100).reset_index()
cluster_percentage.columns = ['Cluster', 'Percentage']
cluster_percentage.sort_values(by='Cluster', inplace=True)

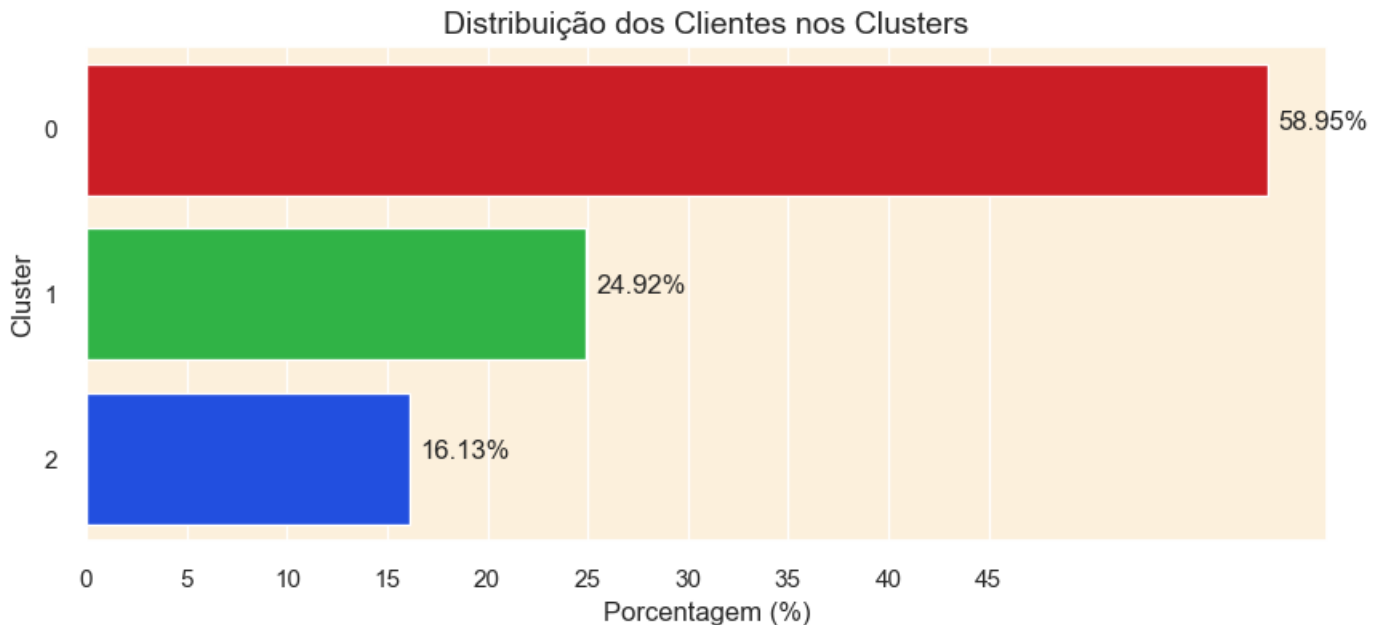
# Criar um gráfico de barras horizontais
plt.figure(figsize=(10, 4))
sns.barplot(x='Percentage', y='Cluster', data=cluster_percentage, orient='h',
palette=colors)

# Adicionar porcentagens nas barras
for index, value in enumerate(cluster_percentage['Percentage']):
```

```
plt.text(value + 0.5, index, f'{value:.2f}%')

plt.title('Distribuição dos Clientes nos Clusters', fontsize=14)
plt.xticks(ticks=np.arange(0, 50, 5))
plt.xlabel('Porcentagem (%)')

# Mostrar o gráfico
plt.show()
```



Inferências:

A distribuição dos clientes entre os clusters, conforme mostrado pelo gráfico de barras, sugere uma distribuição bastante equilibrada, com os clusters 1 e 2 contendo cerca de 16 e 24% dos clientes cada e o cluster 0 acomodando a maioria com mais de 58% dos clientes.

Além disso, o fato de nenhum cluster conter uma porcentagem muito pequena de clientes nos assegura que cada cluster é significativo e não representa apenas outliers ou ruídos nos dados. Essa configuração permite uma análise mais detalhada e informada de diferentes segmentos de clientes, facilitando a tomada de decisões eficazes e informadas.

Passo 10.3 | Métricas de Avaliação

Para avaliar ainda mais a qualidade do nosso agrupamento, vou empregar as seguintes métricas:

- **Pontuação de Silhueta:** Uma medida para avaliar a distância de separação entre os clusters. Valores mais altos indicam uma melhor separação dos clusters. Varia de -1 a 1.
- **Pontuação de Calinski Harabasz:** Esta pontuação é usada para avaliar a dispersão entre e dentro dos clusters. Uma pontuação mais alta indica clusters mais bem definidos.
- **Pontuação de Davies Bouldin:** Avalia a similaridade média entre cada cluster e seu cluster mais semelhante. Valores mais baixos indicam uma melhor separação dos clusters.

```
# Calcular o número de clientes
```

```

num_observations = len(customer_data_pca)

# Separar as características e os rótulos dos clusters
X = customer_data_pca.drop('cluster', axis=1)
clusters = customer_data_pca['cluster']

# Calcular as métricas
sil_score = silhouette_score(X, clusters)
calinski_score = calinski_harabasz_score(X, clusters)
davies_score = davies_bouldin_score(X, clusters)

# Criar uma tabela para exibir as métricas e o número de observações
table_data = [
    ["Número de Observações", num_observations],
    ["Pontuação de Silhueta", sil_score],
    ["Pontuação de Calinski Harabasz", calinski_score],
    ["Pontuação de Davies Bouldin", davies_score]
]

# Exibir a tabela
print(tabulate(table_data, headers=["Métrica", "Valor"], tablefmt='pretty'))

```

Métrica	Valor
Número de Observações	4153
Pontuação de Silhueta	0.3685548828288789
Pontuação de Calinski Harabasz	2496.027311846308
Pontuação de Davies Bouldin	0.9744070846901248

Inferências sobre a Qualidade do Agrupamento:

- A **Pontuação de Silhueta** de aproximadamente 0,368, embora não esteja próxima de 1, ainda indica uma quantidade razoável de separação entre os clusters. Sugere que os clusters são um tanto distintos, mas pode haver pequenas sobreposições entre eles. Geralmente, uma pontuação mais próxima de 1 seria ideal, indicando clusters mais distintos e bem separados.
- A **Pontuação de Calinski Harabasz** é 2496, o que é consideravelmente alto, indicando que os clusters são bem definidos. Uma pontuação mais alta nesta métrica geralmente sinaliza melhores definições de clusters, o que implica que nosso agrupamento conseguiu encontrar uma estrutura substancial nos dados.
- A **Pontuação de Davies Bouldin** de 0,97 é uma pontuação boa, indicando um nível moderado de similaridade entre cada cluster e seu cluster mais semelhante. Uma pontuação mais baixa é geralmente melhor, pois indica menos similaridade entre os clusters, e, portanto, nossa pontuação aqui sugere uma separação decente entre os clusters.

Em conclusão, as métricas sugerem que o agrupamento é de boa qualidade, com clusters bem definidos e razoavelmente separados. No entanto, ainda pode haver espaço para otimização adicional para melhorar a separação e definição dos clusters, potencialmente tentando outros algoritmos de agrupamento e redução de dimensionalidade.

Passo 11 | Análise e Perfilamento dos Clusters

Nesta seção, vamos analisar as características de cada cluster para entender os comportamentos e preferências distintos de diferentes segmentos de clientes e também perfilar cada cluster para identificar os principais traços que definem os clientes em cada cluster.

Passo 11.1 | Abordagem do Gráfico de Radar

Primeiramente, vamos criar gráficos de radar para visualizar os valores dos centróides de cada cluster em diferentes características. Isso pode fornecer uma comparação visual rápida dos perfis de diferentes clusters. Para construir os gráficos de radar, é essencial primeiro calcular o centróide para cada cluster. Esse centróide representa o valor médio para todas as características dentro de um cluster específico. Em seguida, vamos exibir esses centróides nos gráficos de radar, facilitando uma visualização clara das tendências centrais de cada característica em vários clusters:

```
# Configurar a coluna 'CustomerID' como índice e atribuí-la a um novo dataframe
df_customer = customer_data_cleaned.set_index('CustomerID')

# Padronizar os dados (excluindo a coluna de cluster)
scaler = StandardScaler()
df_customer_standardized = scaler.fit_transform(df_customer.drop(columns=['cluster'],
axis=1))

# Criar um novo dataframe com valores padronizados e adicionar a coluna de cluster de volta
df_customer_standardized = pd.DataFrame(df_customer_standardized,
columns=df_customer.columns[:-1], index=df_customer.index)
df_customer_standardized['cluster'] = df_customer['cluster']

# Calcular os centróides de cada cluster
cluster_centroids = df_customer_standardized.groupby('cluster').mean()

# Função para criar um gráfico de radar
def create_radar_chart(ax, angles, data, color, cluster):
    # Plotar os dados e preencher a área
    ax.fill(angles, data, color=color, alpha=0.4)
    ax.plot(angles, data, color=color, linewidth=2, linestyle='solid')

    # Adicionar um título
    ax.set_title(f'Cluster {cluster}', size=20, color=color, y=1.1)

# Definir os dados
labels = np.array(cluster_centroids.columns)
num_vars = len(labels)

# Calcular o ângulo de cada eixo
angles = np.linspace(0, 2 * np.pi, num_vars, endpoint=False).tolist()

# O gráfico é circular, então precisamos "completar o loop" e anexar o início ao final
labels = np.concatenate((labels, [labels[0]]))
angles += angles[:1]

# Inicializar a figura
fig, ax = plt.subplots(figsize=(20, 10), subplot_kw=dict(polar=True), nrows=1, ncols=3)

# Criar gráfico de radar para cada cluster
for i, color in enumerate(colors):
    data = cluster_centroids.loc[i].tolist()
    data += data[:1] # Completar o loop
```

```

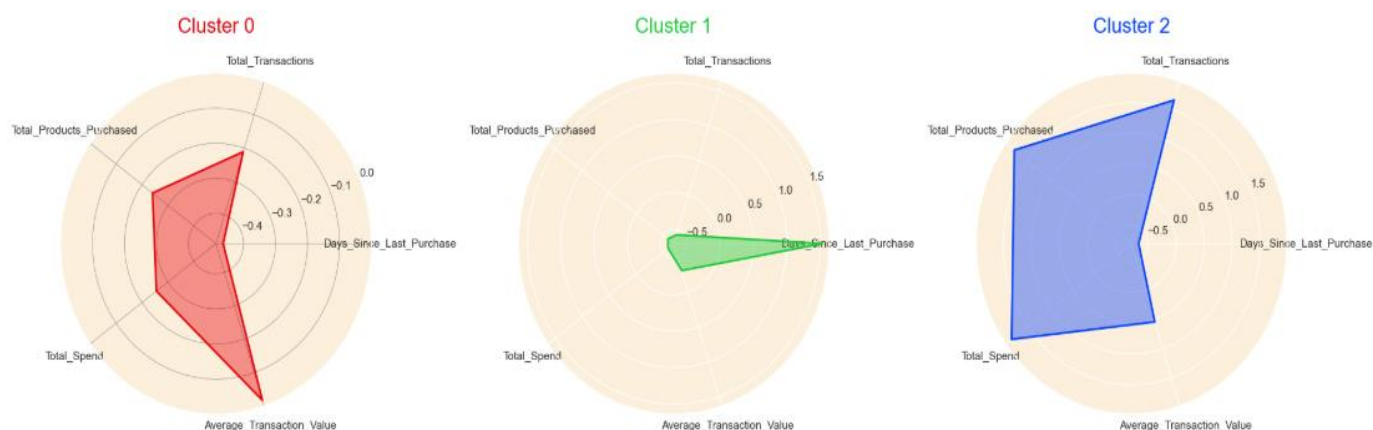
create_radar_chart(ax[i], angles, data, color, i)

# Adicionar dados de entrada
ax[0].set_xticks(angles[:-1])
ax[0].set_xticklabels(labels[:-1])
ax[1].set_xticks(angles[:-1])
ax[1].set_xticklabels(labels[:-1])
ax[2].set_xticks(angles[:-1])
ax[2].set_xticklabels(labels[:-1])

# Adicionar uma grade
ax[0].grid(color='grey', linewidth=0.5)

# Exibir o gráfico
plt.tight_layout()
plt.show()

```



Perfis dos Clusters Derivados da Análise do Gráfico de Radar

Cluster 0 (Gráfico Vermelho):

Perfil: **Clientes Inativos ou Ocasionais**

- Recência (Days_Since_Last_Purchase):
 - Valor mais alto entre os clusters → Clientes que não compram há mais tempo (menos ativos recentemente).
- Frequência (Total_Transactions):
 - Valor mais baixo → Menor número de transações por cliente.
- Valor Monetário (Total_Spend e Average_Transaction_Value):
 - Gastos totais e médios mais baixos → Clientes de baixo valor.
- Total_Products_Purchased:
 - Quantidade de produtos comprados abaixo da média.
- **ISSO INDICA:**
 - Compram raramente, gastam pouco e estão distantes da última compra.
 - Estratégia: Campanhas de reativação (ex.: descontos para retorno) ou investigar motivos de abandono.

Cluster 1 (Gráfico Verde):

Perfil: **Clientes de Alto Valor, mas Pouco Frequentes**

- Recência:
 - Valor intermediário → Alguma atividade recente, mas não tão frequente quanto o Cluster 2.
- Frequência:
 - Número de transações moderado.
- Valor Monetário:
 - Gastos médios altos (Average_Transaction_Value elevado) → Clientes que compram itens caros, mas em menor volume.
- Total_Products_Purchased:
 - Quantidade de produtos próxima à média.
- **ISSO INDICA:**
 - Fazem compras ocasionais, mas com alto ticket médio (ex.: compram produtos premium).
 - Estratégia: Fidelização com programas VIP ou bundles de produtos premium.

Cluster 2 (Gráfico Azul):

Perfil: **Clientes Fiéis e Ativos**

- Recência:
 - Valor mais baixo → Clientes que compraram recentemente.
- Frequência:
 - Número de transações mais alto → Compram com frequência.
- Valor Monetário:
 - Gastos totais altos (Total_Spend elevado), mas ticket médio moderado → Volume alto de compras.
- Total_Products_Purchased:
 - Quantidade de produtos significativamente maior → Clientes que compram em grande volume.
- **ISSO INDICA:**
 - Compram frequentemente, em grande quantidade, mas não necessariamente itens caros.
 - Estratégia: Programas de recompensa (ex.: pontos por volume) ou ofertas de produtos complementares.

Passo 11.2 | Abordagem do Gráfico de Histograma

Para validar os perfis identificados a partir dos gráficos de radar, podemos plotar histogramas para cada característica segmentada pelos rótulos dos clusters. Esses histogramas nos permitirão inspecionar visualmente a distribuição dos valores das características dentro de cada cluster, confirmando ou refinando os perfis que criamos com base nos gráficos de radar.

```
# Plotar histogramas para cada característica segmentada pelos clusters
features = customer_data_cleaned.columns[1:-1]
clusters = customer_data_cleaned['cluster'].unique()
clusters.sort()

# Configurar os subplots
n_rows = len(features)
n_cols = len(clusters)
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 3 * n_rows))

# Plotar histogramas
for i, feature in enumerate(features):
    for j, cluster in enumerate(clusters):
        data = customer_data_cleaned[customer_data_cleaned['cluster'] ==
cluster][feature]
        axes[i, j].hist(data, bins=20, color=colors[j], edgecolor='w', alpha=0.7)
        axes[i, j].set_title(f'Cluster {cluster} - {feature}', fontsize=15)
        axes[i, j].set_xlabel('')
        axes[i, j].set_ylabel('')

# Ajustar o layout para evitar sobreposição
plt.tight_layout()
plt.show()
```



Perfis Refinados dos Clusters:

Com base na análise detalhada dos gráficos de radar e histogramas, aqui estão os perfis refinados e títulos para cada cluster:

Cluster 0 (Inativos/Ocasionais)

- Days_Since_Last_Purchase:
 - Distribuição concentrada em valores altos (clientes inativos há meses).
- Total_Transactions e Total_Spend:
 - Valores baixos e concentrados próximos a zero.
- Confirmação: Perfil de baixo engajamento e gasto.

Cluster 1 (Alto Valor, Pouco Frequente)

- Average_Transaction_Value:
 - Cauda longa à direita → Alguns clientes têm gastos médios muito altos (outliers positivos).
- Total_Products_Purchased:
 - Distribuição equilibrada, sem picos extremos.
- Confirmação: Clientes que preferem qualidade sobre quantidade.

Cluster 2 (Fiéis e Ativos)

- Total_Transactions e Total_Products_Purchased:
 - Distribuição deslocada para valores altos → Muitas transações e produtos.
- Total_Spend:
 - Valores altos, mas sem picos extremos (consistência no volume).
- Confirmação: Comportamento de compra recorrente e volumosa.

Conclusão dos Perfis

Cluster	Nome do Perfil	Características-Chave	Estratégia Recomendada
0	Inativos/Ocasionais	Baixa recência, baixo gasto, poucas transações	Reativação (descontos, e-mails personalizados)
1	Alto Valor, Pouco Frequentes	Ticket médio alto, frequência moderada	Fidelização (ofertas premium, atendimento exclusivo)
2	Fiéis e Ativos	Alta frequência, volume alto de compras	Programas de recompensa (pontos, frete grátis)

Passo 12 | Sistema de Recomendação

Na fase final deste projeto, vamos desenvolver um sistema de recomendação para melhorar a experiência de compra online. Este sistema sugerirá produtos aos clientes com base nos padrões de compra predominantes em seus respectivos clusters. Anteriormente no projeto, durante a preparação dos dados dos clientes, isolamos uma pequena fração (5%) dos clientes identificados como outliers e os reservei em um conjunto de dados separado chamado **outliers_data**.

Agora, focando nos 95% principais do grupo de clientes, analisaremos os dados limpos dos clientes para identificar os produtos mais vendidos em cada cluster. Com base nessas informações, o sistema criará recomendações personalizadas, sugerindo **os três principais produtos** populares em seu cluster que eles ainda não compraram. Isso não apenas facilita estratégias de marketing direcionadas, mas também enriquece a experiência de compra pessoal, potencialmente impulsionando as vendas. Para o grupo de outliers, uma abordagem básica pode ser recomendar produtos aleatórios, como um ponto de partida para envolvê-los.

```
# Passo 1: Extrair os CustomerIDs dos outliers e remover suas transações do dataframe principal
outlier_customer_ids = outliers_data['CustomerID'].astype('float').unique()
df_filtered = df[~df['CustomerID'].isin(outlier_customer_ids)]

# Passo 2: Garantir consistência no tipo de dados para CustomerID em ambos os dataframes antes de mesclar
customer_data_cleaned['CustomerID'] = customer_data_cleaned['CustomerID'].astype('float')

# Passo 3: Mesclar os dados de transação com os dados dos clientes para obter as informações de cluster para cada transação
merged_data = df_filtered.merge(customer_data_cleaned[['CustomerID', 'cluster']],
on='CustomerID', how='inner')

# Passo 4: Identificar os 10 produtos mais vendidos em cada cluster com base na quantidade total vendida
best_selling_products = merged_data.groupby(['cluster', 'StockCode',
'Description'])['Quantity'].sum().reset_index()
best_selling_products = best_selling_products.sort_values(by=['cluster', 'Quantity'],
ascending=[True, False])
top_products_per_cluster = best_selling_products.groupby('cluster').head(10)

# Passo 5: Criar um registro dos produtos comprados por cada cliente em cada cluster
customer_purchases = merged_data.groupby(['CustomerID', 'cluster',
'StockCode'])['Quantity'].sum().reset_index()

# Passo 6: Gerar recomendações para cada cliente em cada cluster
recommendations = []
for cluster in top_products_per_cluster['cluster'].unique():
    top_products = top_products_per_cluster[top_products_per_cluster['cluster'] ==
cluster]
    customers_in_cluster = customer_data_cleaned[customer_data_cleaned['cluster'] ==
cluster]['CustomerID']
    for customer in customers_in_cluster:
        # Identificar produtos já comprados pelo cliente
        customer_purchased_products =
customer_purchases[(customer_purchases['CustomerID'] == customer) &
(customer_purchases['cluster']
== cluster)][['StockCode']].tolist()
        # Encontrar os 3 principais produtos na lista de mais vendidos que o cliente
        ainda não comprou
```

```

top_products_not_purchased =
top_products[~top_products['StockCode'].isin(customer_purchased_products)]
top_3_products_not_purchased = top_products_not_purchased.head(3)
# Adicionar as recomendações à lista
recommendations.append([customer, cluster] +
top_3_products_not_purchased[['StockCode', 'Description']].values.flatten().tolist())

# Passo 7: Criar um dataframe a partir da lista de recomendações e mesclá-lo com os dados
originais dos clientes
recommendations_df = pd.DataFrame(recommendations,
                                  columns=['CustomerID', 'cluster', 'Rec1_StockCode',
'Rec1_Description',
                                  'Rec2_StockCode', 'Rec2_Description',
'Rec3_StockCode', 'Rec3_Description'])
customer_data_with_recommendations = customer_data_cleaned.merge(recommendations_df,
on=['CustomerID', 'cluster'], how='right')

# Exibir 10 linhas aleatórias do dataframe customer_data_with_recommendations
customer_data_with_recommendations.set_index('CustomerID').iloc[:, -6:].sample(10,
random_state=0)

```

	Rec1_StockCode	Rec1_Description	Rec2_StockCode	Rec2_Description	Rec3_StockCode	Rec3_Description
CustomerID						
15844.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	84879	ASSORTED COLOUR BIRD ORNAMENT	85123A	WHITE HANGING HEART T-LIGHT HOLDER
14871.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	84879	ASSORTED COLOUR BIRD ORNAMENT	85123A	WHITE HANGING HEART T-LIGHT HOLDER
17924.0	85099B	JUMBO BAG RED RETROSPOT	84879	ASSORTED COLOUR BIRD ORNAMENT	21212	PACK OF 72 RETROSPOT CAKE CASES
18077.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	85099B	JUMBO BAG RED RETROSPOT	84879	ASSORTED COLOUR BIRD ORNAMENT
18224.0	17003	BROCADE RING PURSE	23167	SMALL CERAMIC TOP STORAGE JAR	15036	ASSORTED COLOURS SILK FAN
12517.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	84879	ASSORTED COLOUR BIRD ORNAMENT	85123A	WHITE HANGING HEART T-LIGHT HOLDER
12361.0	85123A	WHITE HANGING HEART T-LIGHT HOLDER	17003	BROCADE RING PURSE	23167	SMALL CERAMIC TOP STORAGE JAR
15308.0	85123A	WHITE HANGING HEART T-LIGHT HOLDER	17003	BROCADE RING PURSE	23167	SMALL CERAMIC TOP STORAGE JAR
13962.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	85123A	WHITE HANGING HEART T-LIGHT HOLDER	85099B	JUMBO BAG RED RETROSPOT
15246.0	17003	BROCADE RING PURSE	23167	SMALL CERAMIC TOP STORAGE JAR	15036	ASSORTED COLOURS SILK FAN